CH 2 — Classes

Luke Lu • 2026-11-10

Intro to Class and Objects

> 📘 **Definition — Classes and Objects**
>
> Imperative Programming
>
> - Procedural Programming: data + standalone functions
> - Object-Oriented Programming: put the data and the functions that operate that data within unit or construct
>   - The construct is called **class**
>   - An instance/value of a class is called an **object**

Example:

A simple class called Student(Program "06-classes/version1.cc/student.cc"):

```
struct Student{
  int assignment, mt, final;
  float grade();
}

float Student::grade(){ //In the scope of student Class, the function grade
   return 0.4 * assignment + 0.2 * mt + 0.4 * final;
}
```

Note: in C++, `struct` can have functions within, and C++ `structs` are classes

Program "06-classes/version2.cc" is the more formal and preferred way of usage.

```
Student billy{90, 55, 45}; //C-style  initialization
//billy is a student object

float res1 = billy.grade(); //grade() has access to billy's marks

Student lily{95, 80, 85};

cout << lily.grade(); //grade() has access to lily's mark
```

> 📘 **Definition — Methods**
>
> A "function" part of a class is called "**methods**", which is commonly used
>
> A **method** can only be called by using an **object**
>
> Within the **method**, the method has access to the fields of that object.
>
> All **methods** have a hidden parameter named **`this`**, which is a pointer to the object use to call the method

Example:

```
billy.grade(); //"this" is equiv to "&billy", or interpret that their address is same
```

For the function, `grade` is actually

```
 float grade(){
  return this->assignment * 0.5 + this->mt * 0.2 + this->final 0.4;
};
```

We assume `this->` if not present and identifier undeclared

Object Initialization

In C++, we can write special methods to initialize objects

Previously,

1. C-style Initialization if there is no explicit constructor
   - Memory allocation
   - Field initialization in declaration order with parameter values in declaration order
     - In case of under-initialization, system does default initialization

       non-class/primitive data $\Longrightarrow$ assigned to `0`
2. C++ style Initialization if there is at least one explicit constructor
   - Memory allocation
   - Field initialization in declaration order
     - Primitives are not initialized
     - Classes are being called with **default constructors**
   - **Constructor** calls
     - Cannot default initialize in C-style
     - Logging
     - Constructor/function overloading
     - Sanity checks, restriction on input

> 📘 **Definition — Constructors Rules**
>
> - Same name as the class name
> - No return type
> - Default constructor does not have brackets in the class
>
>   ```
>   CON a constructor(); //ILLEGAL
>   CON b; //LEGAL
>   ```

```
struct Student{
  //...
  Student(int assign, int mt, int final){
    this->assign = assign;
    this->mt = mt;
    this->final = final;
  }
}
```

Member-Initialization List:

- field insitialization in declaration order of C++ style for fields named in the list
- use member-initilization list for all initializations
- use constructor for sanity checks, logging, etc.
- MIL processed in field declaration order. Not in MIL order.

```
struct id{
  string name;

  id;
  id(string name){ this->name = name;}
}
struct posn{
  int x, y;
  id uid;
  posn(int x, int y, string name): y{y*2}, x{x+1}, uid{name+ "?"} {}
}
```

For **ONE** parameter constructors:

```
struct Node {
  int data;
  Node *next;
  Node(int data):data{data}, next{nullptr}{}
}
Node m{4};
Node m = 4; //implicit conversion
foo(m);
foo(4); //One parameter constructor create implicit conversions
```

Note: implicit conversion for one parameter constructors could be useful and also be troublesome. We can disable implicit conversion by keyword: `explicit`

```
struct Node {
  int data;
  Node *next;
  explicit Node(int data):data{data}, next{nullptr}{}
}
Node m{4};
Node m = 4; //ILLEGAL
foo(m);
foo(4); //ILLEGAL
```

Class Methods

A class comes with a default constructor and the **Big 5**:

- Copy Constructor
- Copy Assignment Operator
- Destructor
- Move Constructor
- Move Assignment Operator

Definition — Copy Constructor

Copy Constructor creates a copy that is a new instance. It is used

- when constructing an object as a copy of another object
- called when an object is passed by value
- called when an object is return by value

Example:

1. Default implementation

```
//default hidden implementation
Student::Student(const Student &other) mt{other.mt}, final{other.final} {}
//must pass by reference
//Sometime, the default implemenation is not suitable

Student s1{90, 98, 33};
Student s2{s1}; //s2 = s1; the copy constructor
```

2. Custom implementation

```
struct Node {
  int data;
  Node *next;
  Node (int data, Node *next): data{data}, next{next}{}
  Node(const Node &other): data{other.data}, next{other.next}{}

  Node::Node(const Node &other) : \
  data{other.data}, next{other.next?new Node {*other.next}:nullptr;} {}

};

Node *p new Node{1, new Node {2, new Node {3, nullptr}}}
//p->1->2->3->null

Node m{*p}; //copy constructor the default

Node *q = new Node{*p}; //copy constructor
// Before having custom implementation
//m->1->2(of p)->3(of p)->null(of p)
//q->1->2(of p)->3(of p)->null(of p)

// After having custom implementation
//m->1->2->3->null
//q->1->2->3->null
```

📘 **Definition — Destructor**

Every class has **exactly one** destructor

- cannot overload destructor
- takes no parameters
- For a class `T`, its destructor is written as

  `~T(){}`
- no return type
- default destructor body is empty
- sometimes the default destructor is not correct

When an object is destroyed, the destructor is called

When are objects destroyed:

- When a stack allocated object goes out of scope
- When call `delete` on the address of a healp allocated object

Steps for object destruction

1. Destructor body runs
2. Fields that are objects are destroyed (by calling their destructor)
3. Spaces are reclaimed

Example:

```
Node *p  = new Node{1, new Node{2, new Node{3, nullptr}}}; //p->1->2->3->Null

//If we do nothing, we leak 3 node memory
//Note p is not leaked b/c it is in stack as it stores the address to the first node.
//If we write `delete p;`, we are destroying node 1 only, leaking node 2, 3


//Destructor

Node::~Node(){delete next;}

delete p;
//calls delete node 1 -> delete node 2 -> delete node 3 -> delete nullptr
//then space for node 3 reclaimed -> reclaimed node 2 -> reclaimed node 1

Node m{1, new Node{2, nullptr}};
// `m` will be deleted automatically and leak no memory because it is on stack
Node n{3, &m};
// `n` will cause error b/c m is on stack, but C++ expects a node in heap memory
```

Definition — Copy Assignment Operator

Assignment operator

- updates and existing object rather than creating a new instance.
- is implemented as a method
- the default implementation does field for field assignment

Example:

1. Copy constructor vs Assignment operator

```
Student s1{50, 12, 32};
Student s2{s1}; //copy constructor
Student s3 = s1; //copy constructor
Student s4{50, 23, 45};
s3 = s2; //copy assignment operator
```

2. Implementation for `Node`

```
n1 = n2; //n1.operator= (n2);
n1 = n2 = n3; //assignment cascades, so operator= must return a Node reference

Node &Node::operator=(const Node &other){
  if(this == &other) return *this; //self-assignment check, avoid dangling pointer
  data = other.data
  delete next;
  next = other.next?new Node{*other.next}:nullptr;
  //if `new` fails to allocate memory, `next` will not be updated
  //this will cause `next` to be a dangling pointer
  //it would be better to have "all or nothing" behaviour which will be discussed
in future
  return *this; //or return others;

//corrected
Node &Node::operator=(const Node &other){
  if(this == &other) return *this;
  Node *temp{next};
  next = other.next?new Node{*other.next}:nullptr;
  data = other.data;
  delete temp;
  return *this; //or return others;
}

//another way, Copy and Swap Idiom
import<utility>; //for swap()
void Node::swap(Node &other){
  using std::swap;
  swap(data, other.data);
  swap(next, other.next);
}
Node &Node::operator=(const Node &other){
  Node temp{other};
  swap(temp); //swap method from Node
  return *this;//DO NOT write delete temp;, temp is in stack
}
```

Definition — Move Constructor

Refer to “lectures/10-rvalue/node.cc”

Calling `oddsOrEvens` results in 6 basic constructor calls

Returning `evens` will result in 3 copy constructors due to copy elision.

Copy Elision: C++ allows(sometimes requires) that extra copies be avoided

An **r-value** reference is a reference to a temporary value on an object that is about to be destroyed.

If we know that the RHS of a copy is a r-value, then instead of copying we could hijack/steal in hopes of being more efficient.

```
Node::Node(Node &&other): data {other.data}, next {other.next} {
  other.next = nullptr;
}
```

After this implementation, it will only call 1 copy constructor.

Definition — Move Assignment Operator

Refer to “lectures/10-rvalue/node.cc”

```
Node m = ...;
m = oddsOrEvens();

//say other is 1->3->5, and m is ?->?->?->? and the goal is to make m be 1->3->5

Node &Node::operator=(Node &&other){
  using std::swap;
  swap(data, other.data);
  swap(next, other.next);
  return *this;
}
```

If a move constructor or a move assignment operator has been implemented, the compiler will choose it whenever the RHS is a r-value or a l-value that is about to be destroyed.

If not, it will use the copy constructor and copy assigntment operator.

Copy/Move Elision

C++ allows/requires copy/move operations even if this changes observable behaviours.

```
Vec makeVec(){return {4, 5};}

Vec v = makeVec();// one basic constructor call, no copy/move constructor


void foo(Vec v){...}

foo(makeVec()); //one basic constructor, no copy/move constructor
```

Important to know that copy or move elision can happen, not required to memorize situations if when it is required or optional

> 💡 **Tip — Rule of Five**
>
> If implement a custome version of any of the Big Five, custom versions of all five are typically needed.

Operator overloading

C++ requires some operators to be overloaded as methods:

- `operator=`
- `operator[]`
- `operator->`
- `operator()`
- `operator T()`

It is best to implement I/O operator as standalone functions

```
struct Vec{
  int x;
  int y;

  Vec operator+(const Vec &rhs){
  // `this` represents the LHS
  return {x+rhs.x, y+rhs.y};
  }

  Vec operator*(const int k){
    return {k * x, k * y};
  }
}
//Standalone function because `int` is not class `Vec`
Vec operator*(const int k, const Vec &rhs){
  return {rhs * k};
}

Vec v1;
Vec v2;
Vec v3 = v1+v2;
Vec v4 = v1 * 3;
Vec v5 = 3 * v1;//must be implemented as a standalone function
```

Arrays of Object

```
struct Vec{
  int x, y;
  Vec(int x, int y);

};

//No default constructor since we wrote 2 parameter constructor
Vec arr1[10];
Vec *arr2 = new Vec[10];
//This will not compule since when an array of objects is constructed, each object is
default constructed(by default constructor)
```

Option1: Provide a default constructor.

Option2: Stack array: initialize each object explicitly: `Vec arr1[] = {Vec(1, 2), ...};`

Option3(Most Common): Create array of pointers to objects:

```
Vec *arr1[10] //Stack array

vec **p = new Vec *[10]; //Heap array
```

Constant objects and Methods

```
struct Student {
  int assign, mt, final;
  //allowed to mutate in constant methods
  mutable int numcalls;

  float grade() {
    ++ numcalls;
    return assign*0.4+mt*0.2+final*0.4;
    }

};

const Student s1{80, 90, 90};
//cannot modify the fields of s1

s1.grade();
//ILLEGAL, s1 is constant and `grade` would have access the fields of s1 via `this`

//Instead mark the method constant using the `const` keyworld
//Promised not to change the field of `*this`, the compiler will confirm later

float Student::grade() const {
  ++numcalls;
  return assign*0.4+mt*0.2+final*0.4;
  }

//Suppose we want to add a varaible called `numcalls` that records the number of
calls of `grade`, now `grade` is no longer valid because mutation in const methods.

//We use `mutatable` in definition of variable
```

Static Fields and Methods

📘 **Definition — Static**

A **static** field is associated with the class, not instance of the class

A **static method** does not need an instance of the class to be called.
- there is no "`this`" pointer within the static funciton

A **static function** can call other static functions in the same static field.

```
struct Student {
  int assign, mt, final;
  static int numobjs;
  Student(assign, mt, final) assign{assign}, mt{mt}, final{final}{
    ++numobjs;
    }

  static int getStudentCount(){return numobjs;}
};

int main(void){
  Student s1{...}, s2{...};
  cout <<Student::numobs; //prints 2

  cout<<getStudentCount(); //prints 2
}
```

Ways of initializing `numobjs`

1. In "student-impl.cc", `static int numobjs = 0;`
2. initialize in the interface with `inline static int num objs = 0;`

Compare Objects/Values

```
if (s1 < s2){...}
else if(s2 > s1){...}
else{...}
```

In C, we only need one $O(n)$comparison `int n = strcmp(s1, s2)`, but two $O(n)$ comparisons in C++

Now C++ supports `<=>` operator

A comparison is defined for "`std::strong-ordering`" and the value `_::less/equal/greater` with `_::less == 0` is `true`

We can replace "`auto`" to **automatically** infer to the type of the object.

```
import <compare>;

// std::strong-ordering n = s1 <=> s2;
auto n = s1 <=>s2;

if (n < 0){//s1 is smaller than s2}
else if(n> 0){//s1 is greater the s2}
else{//s1 and s2 are equals}

//attempts to answer the question `s1` is greater than, less than, equal, or
unordered compared to `s2` when they are strings
```

```
struct Vec{
  int x, y;
  std::strong-ordering operator<=>(const Vec&other) const{
    auto result = x <=> other.x;
    if(result !=0){return result;}
    return result y <=> other.y;
```

```
  }
}
```

When implement `<=>` , we get `<`, `<=`, `>`, `>=`, `>` for free.

When implement `==` as well, you get `!=` for free.

We get `<=>` for free, but it does ordered lexiographic comparison of fields, like the "`Vec`" implementation.

We can signal to the compiler that we are using the "`default`", and we get all comparison operators for free including "`==`".

```
struct Vec {
  //...
  auto operator<=>(const Vec&other)= default;
};


struct Node{
  int data;
  Node *next;

  std::strong-ordering operator<=>(const Node &other) const{
    auto n = data <=> other.data;
    if(n != 0) return n;
    if (!next && !other.next) return n;
    if(!next) return strong-ordering::less;
    if(!other.next) return strong-ordering::greater;
    return *next <=> *other.next;
  }
};
```

Class Invariants

```
struct Node{
  int data;
  Node *next;
  Node(int data, Node *next)
  ~Node{delete next;}
};

void foo(){
  Node n1{1, new Node {7, nullptr}};
  Node n2{2, n1.next}; // n1.next might have been already destroyed
  Node n3{3, &n2}; //ERROR attempt to free stack object
}
```

Node class assumed that `next` is either `nullptr` or points into the heap and Nodes are not shared.

Some assumption/requirements that needs to hold true for the progrma to work as expected

Examples:

Binary Search Tree invariants:

- Left child node has value less than root
- Right child node has value greater than root

Stack invariants:
- Last in First out

It is difficult to reason about program correctness if we cannot guarantee invariants.

Encapsulation

> 📘 **Definition — Encapsulaiton**
>
> - Treats objects as black boxes as capsules
> - Hide implementation details
> - Expose some selected methods that clients can use

Key words: `public, private, class, protected, friend` are used for Encapsulaiton

> 💡 **Tip — General Advise**
>
> 1. Having one `public` section and one `private` section
> 2. Keep fields `private` and select some methods to be `public` for client

Example

```
struct Vec{
  Vec(int, x, int y);

  private:
    int x, y;

  public:
    Vec operator+(const Vec &other){
      return {x+other.x, y+other.y};
    }

};

int main(){
  Vec v{1, 2}; //LEGAL
  Vec v1 = v+v; //LEGAL constructor and + are both public
  cout << v1.x << v1.y; //ILLEGAL, attempt accessing private fields
}
```

Ideally, we should have default visibility of `private` so that we consciously decide what to make public, but since C++ inherited `struct` from C which have default visibility being `public`.

Classes created with keyword `class` have default visibility of `private`.

> ⚠️ **Warning — Classes**
>
> `struct` defined ARE still CLASSES with default visibility being `private`.

Examples:

Refer back to the `Vec` class

```
class Vec{
  int x, y;
  public:
    Vec(int x, int y);
    Vec operator+(const Vec &other){return {x+other.x, y+other.y};}
};
```

Node Encapsulation

- Wrap Node objects within a list class
- Clients never get access to Nodes

```
//list.cc
export module list;
export class List{
  struct Node; //private nested classes declaration
  Node *theList = nullptr;
  public:
    void addToFront(int i);
    int &ith(int i);
    ~List();
};

//list-impl.cc
module list;
struct List::Node{
  int data;
  Node *next;
  Node(int data, Node next): data{data}, next{next} {}
  ~Node(){delete next;}
};

void List::addToFront(int i){
  theList = new Node(i, theList);
}

List::~List(){delete theList;}
int &List::ith(int i){//assumption that at least i nodes in list
  Node *cur = theList;
  for(int j = 0; j<i &&cur; ++j, cur = cur.next);
  return cur->data;
}

//List::Node::~List{delete theList;} //LEGAL;
```

List traversal

```
for(int j = 0; j < listLength; ++j){
  cout <<list.ith(j); //another for-loop for `ith` method.
}
//O(n^2)
```

We want `O(n)` traversal

```
Node *cur = theList; //Start
while(cur != nullptr){//Compare, End
  ...=cur->data;//Access
```

```
  cur = cur->next;//Update
}


int *begin = arr;//Begin
int *end = arr+arr_len;//End
for(int *p = begin; p!= end; ++p){//Compare, Update
  cout <<*p;//Access
}
```

> 💡 **Tip — Iterator Design Patter**
>
> - $O(n)$ treversal of an encapsulated class
> - Always keep track of where we are within the iteration
> - Always requires Begin, End, Compare, Access, and Update

We need

```
List::begin();//Begin
List::end();//End
List::Iterator();
Iterator::operator++; //Update
Iterator::operator *(); //Access
Iterator::operator!=(); //Compare
```

To use **Ranged-Based For-Loops** in C++

Refer to source “17-iterator/listiter.cc” and “17-iterator/listiter-impl.cc”

```
//list.cc
export module list;
export class List{
  struct Node; //private nested classes declaration
  Node *theList = nullptr;
  public:
    void addToFront(int i);
    int &ith(int i);
    ~List();
    class Iterator{
      Node *cur;
      public:
        explicit Iterator(Node *curr) cur{cur}{}
        Iterator &operator++(){
          cur = cur->next;
          return *this;
        }

        int &operator*(){return cur->data;}

        bool operator!=(const Iterator &other){
          return cur != other.cur;
        }
    };//End of Iterator

    Iterator begin(){return Iterator{theList};}
    Iterator end(){return Iterator{nullptr};}
```

```
};


//list-impl.cc
module list;
struct List::Node{
  int data;
  Node *next;
  Node(int data, Node next): data{data}, next{next} {}
  ~Node(){delete next;}
};

void List::addToFront(int i){
  theList = new Node(i, theList);
}

List::~List(){delete theList;}
int &List::ith(int i){//assumption that at least i nodes in list
  Node *cur = theList;
  for(int j = 0; j<i &&cur; ++j, cur = cur.next);
  return cur->data;
}

//List::Node::~List{delete theList;} //LEGAL;
```

After the implementation, we can use for loop to make iteration to be $O(n)$ while fufilling encapsulation.

```
int main(){
  List lst;
  lst.addToFront(1);
  lst.addToFront(2);
  lst.addToFront(3);
// lst -> 3->2->1->nullptr;

for(List::Iterator it = lst.begin(); it != lst.end(); ++it){
  int n = *it;
  cout << n <<endl;
  }

}
```

We decided to overload operators to utilize C++ “Range-Based” for-loops.

📘 **Definition — Ranged-Based For-loops**

A class `c1` can bs used in a ranged-based for-loop if

- `c1` implements methods `begin(), end()` that return `c2` object
- `c2` implements `operator*, operator++, operator!=`

```
for(auto n:lst){
  cout << n << endl;
}
// n is a by value copy of the returned from *it in each iteration

for(auto &n:lst){
  n = 10;
}
// n is a reference to value returned by *it in each iteration
```

Problem:

Even with this, another problem is that default constructor of `Iterator` is `public`. We want to enforce the user to use only `begin(), end()`.

We cannot directly put it the default constructor to `private` even though `List::Iterator` is a nested class, it cannot access `private` fields/methods of `Iterator`. `begin(), end()` cannot be initialized b/c they are in `List`

Solution:

We use keyword `friend` as it allows `List` to access `private` fields/methods of `Iterator`.

`friend` is one directional, `List` can access private of `Iterator` but the other way is not.

Friendship weakens encapsulation

`List` could break `Iterator` invariant, thus have as few as `friends` as possible.

```
export class List{
  ...
  public:
    ...
    class Iterator{
      Node *cur;
      explicit Iterator(Node *curr) cur{cur}{}
      friend class List; //LINE ADDED
      ...
      public:
        //operator overloads...
    };
};
```

Accessors and Mutators

Advice: keep fields private, then if we want client to read the data or change it. Then we have accessors(getters) and mutator(setters).

Example:

```
class Vec{
  int x, y;
  public:
    int getX() const {return x;}
    void setX(int x) {x = -x;}
};
```

I/O operators (revisit)

I/O operators

- makes sense to implement as standalone functions
- they need read/write access to fields
- fields are private (problematic)

Solutions:

1. Provide appropriate accessors and mutators, but everyone(including clients) gets access now
2. Make I/O operators `friend` of the class

```
class Vec{
  int x, y;
  public:
    ...
    friend ostream &operator<<(ostream &, const vec &);
    //Vec declares that this function is a friend, IT IS NOT Vec's METHOD
};

ostream &operator<<(ostream &out, const vec &v){
  out << v.x<<v.y; //Can access private fields due to friendship
  return out;
}
```

List ==, <=> (revisit)

(Exam question: implementation/overloadding of spaceship operator)

Lists typically implement a `length` method.

1. Iterate over list and count -> $O(n)$
2. Create a `length` field and keep it up to date -> $O(1)$

If `length` can be obtained at constant time, we might have a way to use a shortcut for equality

- If lengths are not equal, then the lists are not equal

```
std::strong-ordering List::operator<=>(const List &other) const {
  if (!theList && !other.theList) return std::strong-ordering::equal;
  if (!theList) return std::strong-ordering::less;
  if (!other.theList) return std::strong-ordering::greater;
  return *theList <=> *other.theList;
  // call to Node spaceship operator, not recursion because we dereferenced
}
bool List::operator==(const List &other) const {
  if (length != other.length) return false;
  return (*this <=> *other) == 0;
}
```

When implement your own <=>, we do not get == anymore.

MIDTERM CUTOFF